

# 基于YOLOv11的目标检测智能体完整技术方案

## 一、项目概述

### 1.1 项目定位

从零构建一个智能体驱动的遥感目标检测平台。用户通过自然语言与智能体交互，智能体自动理解意图、调用YOLOv11检测模型、分析结果并生成专业报告。

### 1.2 核心能力

| 能力        | 说明                         |
|-----------|----------------------------|
| 自然语言交互    | 用户通过对话完成检测、查询、分析等任务        |
| 智能意图识别    | 自动识别用户是要检测图像、查询历史、还是分析数据   |
| 多Agent协作  | Supervisor调度3个专业Agent，各司其职 |
| YOLOv11检测 | 单图/批量/视频/摄像头全场景目标检测        |
| 流式响应      | 实时展示Agent思考过程和检测结果         |
| 知识问答      | 基于遥感领域知识库的RAG问答            |

### 1.3 检测目标

| 类别                    | 说明        |
|-----------------------|-----------|
| 飞机 (airplane)         | 遥感图像中的飞行器 |
| 油罐 (storage-tank)     | 储油/储气罐    |
| 立交桥 (overpass)        | 道路立交桥     |
| 操场 (baseball-diamond) | 运动场地      |

## 二、技术栈

### 2.1 后端技术栈

| 层级        | 技术组件             | 版本               | 用途说明                        |
|-----------|------------------|------------------|-----------------------------|
| Web框架     | FastAPI          | 0.104+           | 异步API服务                     |
| Agent框架   | LangChain        | 0.3+             | Agent基础能力、工具定义、Prompt管理     |
| 多Agent编排  | LangGraph        | 0.2+             | 多Agent workflow编排、状态管理、路由分发 |
| LLM服务     | OpenAI API       | 1.0+             | GPT-4o-mini 主力模型            |
| 本地LLM     | Ollama           | -                | 本地大模型备选（离线可用）               |
| 目标检测      | YOLOv11          | Ultralytics 8.3+ | 目标检测模型                      |
| 图像处理      | OpenCV           | 4.8+             | 图像读取与处理                     |
|           | Pillow           | 10.0+            | 图像格式转换                      |
| 数据库       | PostgreSQL       | 15+              | 主数据库                        |
|           | SQLAlchemy       | 2.0+             | ORM框架                       |
| 向量存储      | Pgvector         | -                | PostgreSQL向量扩展              |
| Embedding | OpenAI Embedding | -                | 文本向量化                       |
| 缓存        | Redis            | 7+               | 缓存 + 对话历史存储                 |
| 对象存储      | MinIO            | 7.2+             | 图片/模型存储                     |
| 认证安全      | JWT              | python-jose      | 用户认证                        |
|           | bcrypt           | passlib          | 密码加密                        |
| 数据验证      | Pydantic         | 2.5+             | 请求/响应数据验证                   |

## 2.2 前端技术栈

| 层级       | 技术组件         | 版本   | 用途说明              |
|----------|--------------|------|-------------------|
| 框架       | Vue 3        | 3.5+ | 前端框架              |
| 构建       | Vite         | 6.0+ | 构建工具              |
| UI组件     | Element Plus | 2.7+ | UI组件库             |
| 状态管理     | Pinia        | 3.0+ | 状态管理              |
| 路由       | Vue Router   | 4.6+ | 路由管理              |
| 图表       | ECharts      | 5.5+ | 数据可视化             |
| HTTP     | Axios        | 1.7+ | HTTP客户端           |
| Markdown | markdown-it  | 14+  | Agent回复Markdown渲染 |
| 流式输出     | fetch stream | -    | SSE流式响应           |

2.3 基础设施

| 组件    | 技术选型                        | 用途      |
|-------|-----------------------------|---------|
| 容器化   | Docker + Docker Compose     | 应用容器化部署 |
| LLM服务 | Ollama（本地） / OpenAI API（云端） | 大模型推理   |

三、项目目录结构设计

3.1 整体结构

```
rsod-agent-platform/
├─ backend/           # 后端服务
├─ frontend/         # 前端服务
├─ models/           # YOLOv11模型文件
├─ docs/             # 项目文档
├─ docker-compose.yml # 容器编排
├─ .env              # 环境变量
└─ .gitignore
```

3.2 后端目录结构

```
backend/
├─ main.py           # 应用入口
├─ requirements.txt  # Python依赖
├─ Dockerfile        # 后端容器
├─ app/
│   └─ __init__.py
```

```
├── config/                                # 配置模块
│   ├── __init__.py
│   ├── settings.py                       # 全局配置
│   ├── detection.py                      # 检测配置
│   └── llm.py                            # LLM配置
├── entity/                               # 数据模型
│   ├── __init__.py
│   ├── db_models.py                     # SQLAlchemy数据库模型
│   └── schemas.py                       # Pydantic请求/响应模型
├── database/                             # 数据库
│   ├── __init__.py
│   └── session.py                       # 数据库连接与会话
├── storage/                              # 存储
│   ├── __init__.py
│   └── minio_client.py                  # MinIO客户端
├── services/                             # 业务服务
│   ├── __init__.py
│   ├── detection_service.py             # YOLOv11检测服务
│   ├── user_service.py                  # 用户服务
│   └── history_service.py                # 历史记录服务
├── agent/                                # 智能体模块
│   ├── __init__.py
│   ├── graph.py                         # LangGraph状态图（核心编排）
│   ├── supervisor.py                    # Supervisor Agent（路由调度）
│   ├── state.py                         # AgentState状态定义
│   ├── prompts.py                       # 所有Agent的Prompt模板
│   ├── memory.py                        # 对话记忆管理（Redis）
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── detection_agent.py           # 检测Agent
│   │   ├── analysis_agent.py            # 分析Agent
│   │   └── qa_agent.py                  # 问答Agent
│   └── tools/
│       ├── __init__.py
│       ├── detection_tool.py             # 检测工具
│       ├── analysis_tool.py              # 分析工具
│       └── knowledge_tool.py             # 知识库工具
├── rag/                                  # RAG知识库模块
│   ├── __init__.py
│   ├── document_loader.py                # 文档加载与分块
│   ├── embedding.py                      # 向量化
│   └── retriever.py                      # 检索器
├── vectorstore/                          # 向量存储
│   └── __init__.py
```

```

├── pgvector_client.py      # Pgvector客户端
├── api/                   # API路由
│   ├── __init__.py
│   ├── agent.py           # 智能体API
│   ├── detection.py       # 检测API
│   ├── auth.py            # 认证API
│   └── history.py         # 历史记录API
├── tests/                 # 测试
│   ├── test_agent.py
│   ├── test_detection.py
│   └── test_api.py

```

### 3.3 前端目录结构

```

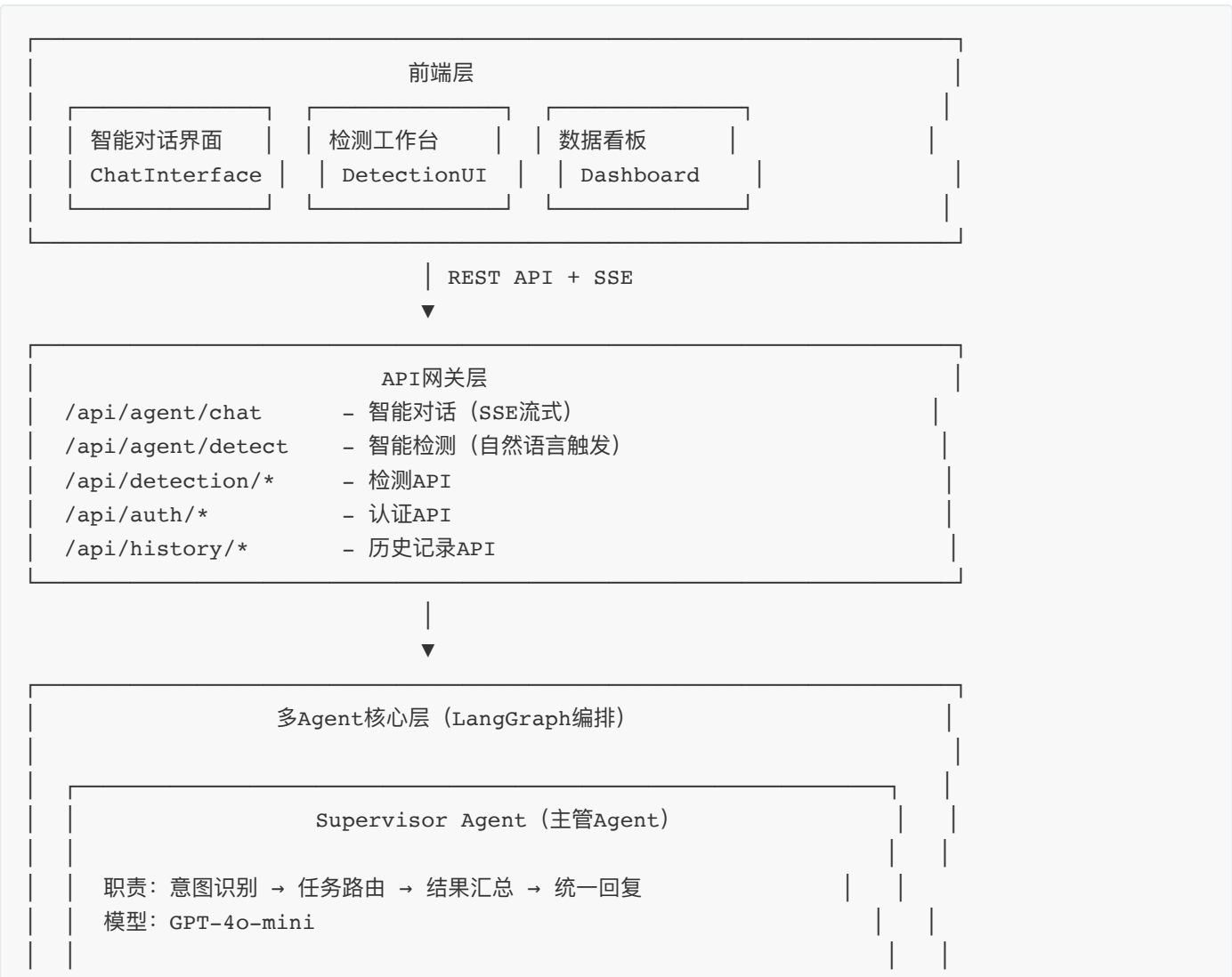
frontend/
├── package.json
├── vite.config.js
├── index.html
├── Dockerfile
├── nginx.conf
├── src/
│   ├── main.js            # 入口
│   ├── App.vue            # 根组件
│   ├── router/
│   │   └── index.js       # 路由配置
│   ├── stores/
│   │   ├── user.js        # 用户状态
│   │   └── agent.js       # 智能体状态
│   ├── api/
│   │   ├── request.js     # Axios封装
│   │   ├── agent.js       # 智能体API
│   │   ├── detection.js   # 检测API
│   │   └── auth.js        # 认证API
│   ├── views/
│   │   ├── LoginPage.vue  # 登录页
│   │   ├── RegisterPage.vue # 注册页
│   │   ├── AgentPage.vue  # 智能体对话页（核心）
│   │   ├── DetectionPage.vue # 检测工作台
│   │   ├── HistoryPage.vue # 历史记录
│   │   └── DashboardPage.vue # 数据看板
│   ├── components/
│   │   ├── chat/
│   │   │   ├── ChatInterface.vue # 智能对话界面
│   │   │   └── MessageItem.vue   # 消息气泡

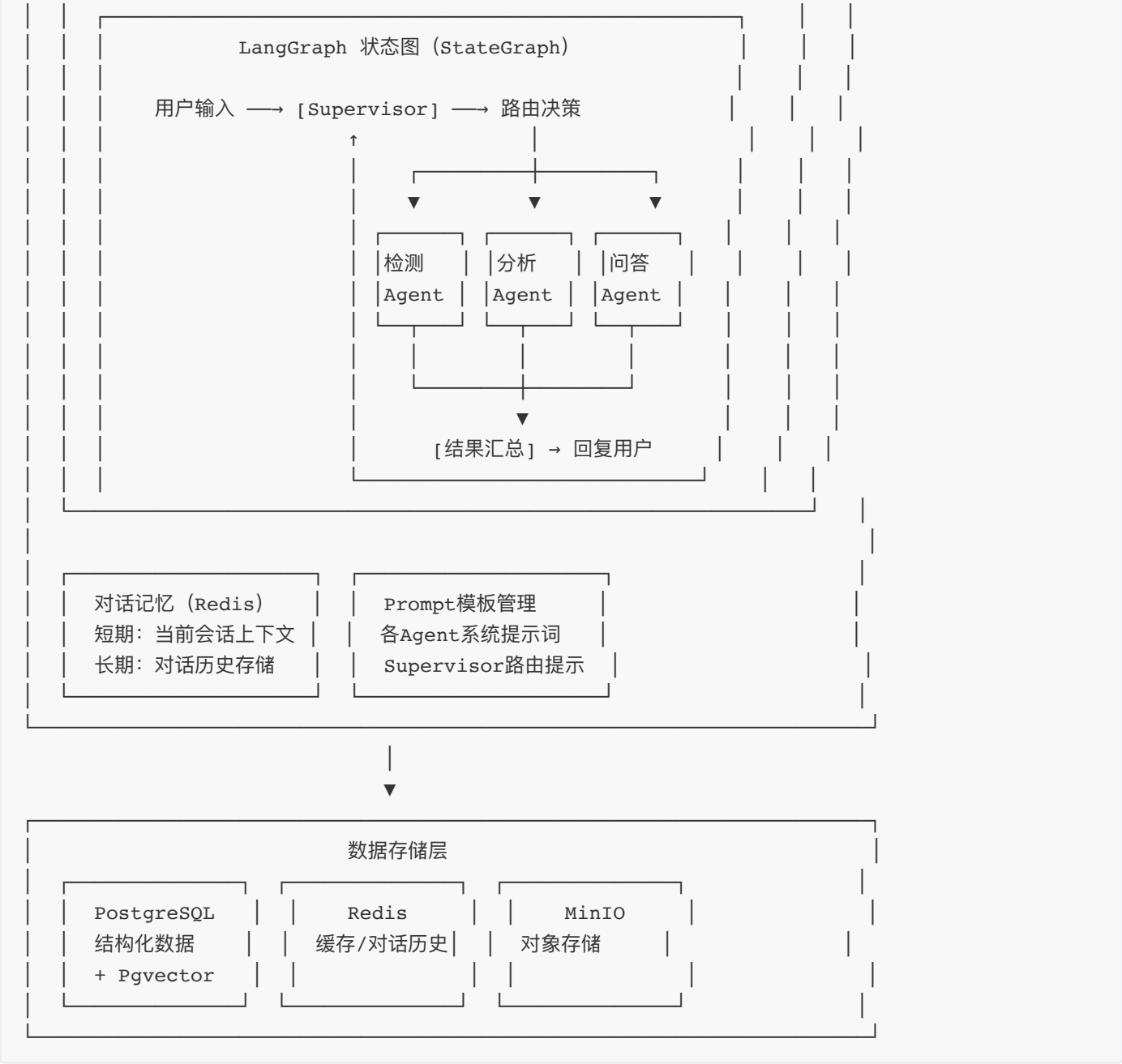
```

```
├── ToolCallDisplay.vue # 工具调用展示
├── detection/
│   ├── ImageUploader.vue # 图像上传
│   ├── DetectionResult.vue # 检测结果展示
│   └── BoundingBox.vue # 检测框绘制
├── common/
│   ├── AppHeader.vue # 顶部导航
│   └── AppSidebar.vue # 侧边栏
├── utils/
│   ├── stream.js # SSE流式处理
│   ├── markdown.js # Markdown渲染
│   └── auth.js # 认证工具
├── assets/
│   └── styles/
│       └── global.css # 全局样式
```

## 四、多Agent架构设计

### 4.1 架构总览





4.2 三个专业Agent职责

| Agent          | 职责       | 绑定工具                                                            | 触发场景        |
|----------------|----------|-----------------------------------------------------------------|-------------|
| DetectionAgent | 遥感图像目标检测 | <code>detect_objects()</code> ,<br><code>detect_batch()</code>  | 用户上传图像并要求检测 |
| AnalysisAgent  | 检测数据统计分析 | <code>query_history()</code> ,<br><code>get_statistics()</code> | 用户查询历史、统计分析 |
| QAAgent        | 遥感领域知识问答 | <code>search_knowledge()</code>                                 | 用户提问专业知识    |

4.3 Supervisor路由机制

```
# backend/app/agent/supervisor.py
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage

SUPERVISOR_ROUTING_PROMPT = """你是一个任务调度器。根据用户输入，选择最合适的Agent处理。

可选Agent：
- detection：用户需要检测遥感图像中的目标（飞机、油罐、立交桥、操场）
- analysis：用户需要查询检测历史、统计分析数据
- qa：用户提出遥感领域知识问题

如果是复合任务，返回多个Agent名称，用逗号分隔。
只回复Agent名称，不要回复其他内容。
"""

class SupervisorAgent:
    """主管Agent - 负责意图识别、任务路由、结果汇总"""

    def __init__(self, llm):
        self.llm = llm

    def route(self, state: AgentState) -> dict:
        """路由：识别用户意图，决定交给哪个Agent"""
        messages = [
            SystemMessage(content=SUPERVISOR_ROUTING_PROMPT),
            state["messages"][-1] # 最新用户消息
        ]
        response = self.llm.invoke(messages)
        return {"next_agent": response.content.strip().lower()}

    def decide_next(self, state: AgentState) -> str:
        """条件路由：根据Supervisor决策跳转"""
        next_agent = state.get("next_agent", "qa")
        if "detection" in next_agent:
            return "detection"
        elif "analysis" in next_agent:
            return "analysis"
        else:
            return "qa"

    def summarize(self, state: AgentState) -> dict:
        """汇总：整合各Agent结果，生成最终回复"""
        context_parts = []
        if state.get("detection_result"):
            context_parts.append(f"检测结果: {state['detection_result']}")
        if state.get("analysis_result"):
            context_parts.append(f"分析结果: {state['analysis_result']}")
        if state.get("qa_result"):
            context_parts.append(f"问答结果: {state['qa_result']}")

        summary_prompt = f"""根据以下信息，生成一份简洁专业的中文回复：

```



```
{''.join(context_parts)}
```

要求:

- 语言简洁专业
- 重点突出关键数据
- 如有检测数据, 列出目标数量和类别

```
"""
```

```
    response = self.llm.invoke([HumanMessage(content=summary_prompt)])  
    return {"final_response": response.content}
```

## 4.4 LangGraph状态图定义

```
# backend/app/agent/state.py
```

```
from typing import TypedDict
```

```
class AgentState(TypedDict):
```

```
    """多Agent共享状态"""
```

```
    messages: list           # 对话消息列表  
    next_agent: str          # 路由目标Agent  
    detection_result: dict    # 检测结果  
    analysis_result: dict     # 分析结果  
    qa_result: str           # 问答结果  
    final_response: str       # 最终回复
```

```
# backend/app/agent/graph.py
```

```
from langgraph.graph import StateGraph, END
```

```
from app.agent.state import AgentState
```

```
from app.agent.supervisor import SupervisorAgent
```

```
from app.agent.agents.detection_agent import DetectionAgent
```

```
from app.agent.agents.analysis_agent import AnalysisAgent
```

```
from app.agent.agents.qa_agent import QAAgent
```

```
def build_agent_graph(llm):
```

```
    """构建多Agent协作状态图"""
```

```
    supervisor = SupervisorAgent(llm)  
    detection = DetectionAgent(llm)  
    analysis = AnalysisAgent(llm)  
    qa = QAAgent(llm)
```

```
    workflow = StateGraph(AgentState)
```

```
    # 添加节点
```

```
    workflow.add_node("supervisor", supervisor.route)  
    workflow.add_node("detection", detection.invoke)  
    workflow.add_node("analysis", analysis.invoke)  
    workflow.add_node("qa", qa.invoke)  
    workflow.add_node("summarize", supervisor.summarize)
```

```
    # 设置入口
```

```
workflow.set_entry_point("supervisor")

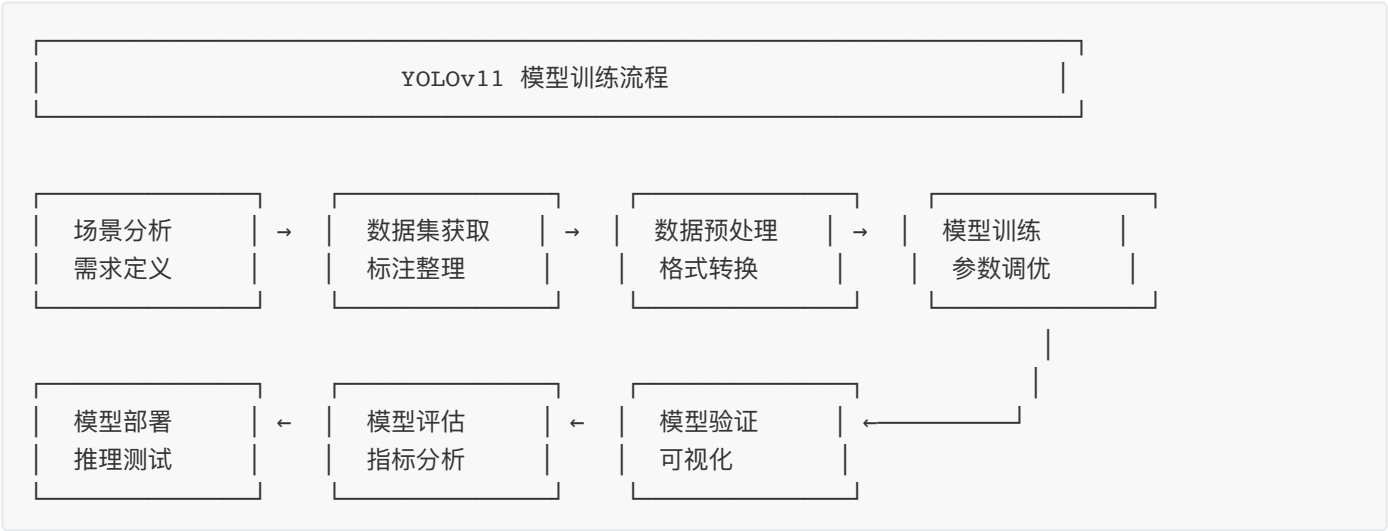
# Supervisor条件路由
workflow.add_conditional_edges(
    "supervisor",
    supervisor.decide_next,
    {
        "detection": "detection",
        "analysis": "analysis",
        "qa": "qa",
    }
)

# 各Agent执行后进入汇总
workflow.add_edge("detection", "summarize")
workflow.add_edge("analysis", "summarize")
workflow.add_edge("qa", "summarize")
workflow.add_edge("summarize", END)

return workflow.compile()
```

## 五、YOLOv11模型训练

### 5.1 训练流程总览



### 5.2 不同场景数据集获取

| 场景   | 数据集来源                     | 类别示例          | 数据量建议        |
|------|---------------------------|---------------|--------------|
| 智慧农业 | Kaggle、Roboflow、自建        | 作物病害、果实成熟度、杂草 | 1000-5000张   |
| 工业检测 | NEU-DET、KolektorSDD、自建    | 划痕、裂纹、凹陷、气泡   | 2000-10000张  |
| 遥感检测 | DOTA、DIOR、RSD、自建          | 飞机、油罐、立交桥、操场  | 5000-20000张  |
| 医疗影像 | Kaggle、Grand Challenge、自建 | 病灶、肿瘤、细胞异常    | 1000-5000张   |
| 交通监控 | UA-DETRAC、BDD100K、自建      | 车辆、行人、交通标志    | 10000-50000张 |
| 安防监控 | WIDER FACE、自建             | 人脸、异常行为、遗留物   | 5000-20000张  |

### 5.3 数据集获取渠道

```
# 1. 公开数据集平台
DATASET_PLATFORMS = {
    "Kaggle": "https://www.kaggle.com/datasets",
    "Roboflow Universe": "https://universe.roboflow.com/",
    "Google Dataset Search": "https://datasetsearch.research.google.com/",
    "Papers with Code": "https://paperswithcode.com/datasets",
    "Visual Data": "https://visualdata.io/",
}

# 2. 遥感专用数据集
REMOTE_SENSING_DATASETS = {
    "DOTA": "https://captain-whu.github.io/DOTA/", # 遥感目标检测
    "DIOR": "https://gcheng-an.github.io/proj/DIOR/", # 遥感图像
    "RSD": "https://github.com/RSIA-LIESMARS-WHU/RSD", # 遥感场景
    "xView": "http://xviewdataset.org/", # 卫星图像
}

# 3. 医疗影像数据集
MEDICAL_DATASETS = {
    "Kaggle Medical": "https://www.kaggle.com/search?q=medical+imaging",
    "Grand Challenge": "https://grand-challenge.org/challenges/",
    "TCIA": "https://www.cancerimagingarchive.net/", # 癌症影像
}

# 4. 工业缺陷数据集
INDUSTRIAL_DATASETS = {
    "NEU-DET": "东北大学表面缺陷数据集",
    "KolektorSDD": "电子元件表面缺陷",
    "DAGM": "纹理表面缺陷",
}
```

### 5.4 数据集格式转换

```
# backend/app/training/data_converter.py
```

```

import os
import json
import xml.etree.ElementTree as ET
from pathlib import Path
import shutil

class DataConverter:
    """数据集格式转换器 - 转换为YOLO格式"""

    @staticmethod
    def voc_to_yolo(xml_dir: str, output_dir: str, class_mapping: dict):
        """
        VOC XML格式 → YOLO TXT格式

        Args:
            xml_dir: XML标注文件目录
            output_dir: 输出目录
            class_mapping: 类别映射 {类名: 类别ID}
        """
        os.makedirs(output_dir, exist_ok=True)

        for xml_file in Path(xml_dir).glob("*.xml"):
            tree = ET.parse(xml_file)
            root = tree.getroot()

            # 获取图像尺寸
            size = root.find("size")
            img_width = int(size.find("width").text)
            img_height = int(size.find("height").text)

            # 转换标注
            yolo_annotations = []
            for obj in root.findall("object"):
                class_name = obj.find("name").text
                if class_name not in class_mapping:
                    continue

                class_id = class_mapping[class_name]
                bbox = obj.find("bndbox")

                # VOC格式: xmin, ymin, xmax, ymax
                xmin = float(bbox.find("xmin").text)
                ymin = float(bbox.find("ymin").text)
                xmax = float(bbox.find("xmax").text)
                ymax = float(bbox.find("ymax").text)

                # 转换为YOLO格式: x_center, y_center, width, height (归一化)
                x_center = (xmin + xmax) / 2 / img_width
                y_center = (ymin + ymax) / 2 / img_height
                width = (xmax - xmin) / img_width
                height = (ymax - ymin) / img_height

```

```

        yolo_annotations.append(f"{class_id} {x_center:.6f} {y_center:.6f}
{width:.6f} {height:.6f}")

    # 保存YOLO标注文件
    txt_file = Path(output_dir) / f"{xml_file.stem}.txt"
    with open(txt_file, "w") as f:
        f.write("\n".join(yolo_annotations))

    @staticmethod
    def coco_to_yolo(json_file: str, img_dir: str, output_dir: str):
        """
        COCO JSON格式 → YOLO TXT格式

        Args:
            json_file: COCO标注JSON文件
            img_dir: 图像目录
            output_dir: 输出目录
        """
        with open(json_file, "r") as f:
            coco_data = json.load(f)

        os.makedirs(output_dir, exist_ok=True)

        # 构建类别映射
        category_mapping = {cat["id"]: idx for idx, cat in
            enumerate(coco_data["categories"])}

        # 构建图像ID到标注的映射
        image_annotations = {}
        for ann in coco_data["annotations"]:
            img_id = ann["image_id"]
            if img_id not in image_annotations:
                image_annotations[img_id] = []
            image_annotations[img_id].append(ann)

        # 转换每个图像的标注
        for img_info in coco_data["images"]:
            img_id = img_info["id"]
            img_width = img_info["width"]
            img_height = img_info["height"]

            yolo_annotations = []
            for ann in image_annotations.get(img_id, []):
                class_id = category_mapping[ann["category_id"]]

                # COCO格式: [x_min, y_min, width, height]
                x_min, y_min, bbox_width, bbox_height = ann["bbox"]

                # 转换为YOLO格式
                x_center = (x_min + bbox_width / 2) / img_width
                y_center = (y_min + bbox_height / 2) / img_height
                width = bbox_width / img_width

```

```

        height = bbox_height / img_height

        yolo_annotations.append(f"{class_id} {x_center:.6f} {y_center:.6f}
{width:.6f} {height:.6f}")

    # 保存YOLO标注文件
    img_name = Path(img_info["file_name"]).stem
    txt_file = Path(output_dir) / f"{img_name}.txt"
    with open(txt_file, "w") as f:
        f.write("\n".join(yolo_annotations))

    @staticmethod
    def labelme_to_yolo(json_dir: str, output_dir: str, class_mapping: dict):
        """
        LabelMe JSON格式 → YOLO TXT格式

        Args:
            json_dir: LabelMe JSON文件目录
            output_dir: 输出目录
            class_mapping: 类别映射 {类名: 类别ID}
        """
        os.makedirs(output_dir, exist_ok=True)

        for json_file in Path(json_dir).glob("*.json"):
            with open(json_file, "r") as f:
                labelme_data = json.load(f)

            img_width = labelme_data["imageWidth"]
            img_height = labelme_data["imageHeight"]

            yolo_annotations = []
            for shape in labelme_data["shapes"]:
                if shape["shape_type"] != "rectangle":
                    continue

                class_name = shape["label"]
                if class_name not in class_mapping:
                    continue

                class_id = class_mapping[class_name]
                points = shape["points"]

                # 获取边界框
                xmin = min(p[0] for p in points)
                ymin = min(p[1] for p in points)
                xmax = max(p[0] for p in points)
                ymax = max(p[1] for p in points)

                # 转换为YOLO格式
                x_center = (xmin + xmax) / 2 / img_width
                y_center = (ymin + ymax) / 2 / img_height
                width = (xmax - xmin) / img_width

```

```

        height = (ymax - ymin) / img_height

        yolo_annotations.append(f"{class_id} {x_center:.6f} {y_center:.6f}
{width:.6f} {height:.6f}")

    # 保存YOLO标注文件
    txt_file = Path(output_dir) / f"{json_file.stem}.txt"
    with open(txt_file, "w") as f:
        f.write("\n".join(yolo_annotations))

```

## 5.5 数据集目录结构

```

datasets/
├── {scene_name}/                                # 场景名称, 如: remote_sensing、industrial_defect
│   ├── images/
│   │   ├── train/                               # 训练集图像
│   │   │   ├── img001.jpg
│   │   │   ├── img002.jpg
│   │   │   └── ...
│   │   ├── val/                                 # 验证集图像
│   │   │   ├── img101.jpg
│   │   │   └── ...
│   │   └── test/                                # 测试集图像
│   │       ├── img201.jpg
│   │       └── ...
│   ├── labels/
│   │   ├── train/                               # 训练集标注 (YOLO格式)
│   │   │   ├── img001.txt
│   │   │   ├── img002.txt
│   │   │   └── ...
│   │   ├── val/                                 # 验证集标注
│   │   │   ├── img101.txt
│   │   │   └── ...
│   │   └── test/                                # 测试集标注
│   │       ├── img201.txt
│   │       └── ...
│   └── data.yaml                                # 数据集配置文件

```

## 5.6 数据集配置文件

```

# datasets/remote_sensing/data.yaml
# 数据集配置

# 路径配置 (相对于训练脚本位置)
path: ./datasets/remote_sensing
train: images/train
val: images/val
test: images/test

# 类别配置

```

```

nc: 4 # 类别数量
names:
  0: airplane
  1: storage-tank
  2: overpass
  3: baseball-diamond

# 中文名称映射 (用于显示)
names_cn:
  0: 飞机
  1: 油罐
  2: 立交桥
  3: 操场

```

## 5.7 模型训练服务

```

# backend/app/training/training_service.py
from ultralytics import YOLO
from pathlib import Path
import yaml
import json
from datetime import datetime
import os

class TrainingService:
    """YOLOv11 模型训练服务"""

    def __init__(self):
        self.base_model_path = "models/yolov11n.pt" # 预训练模型
        self.output_dir = "runs/train"

    def train(
        self,
        data_yaml: str,
        model_name: str = "yolov11n",
        epochs: int = 100,
        img_size: int = 640,
        batch_size: int = 16,
        device: str = "0", # GPU设备ID, "cpu"表示使用CPU
        project: str = None,
        name: str = None,
        **kwargs
    ) -> dict:
        """
        训练模型

        Args:
            data_yaml: 数据集配置文件路径
            model_name: 模型名称 (yolov11n/s/m/l/x)
            epochs: 训练轮数
            img_size: 图像尺寸

```



```

        batch_size: 批次大小
        device: 训练设备
        project: 项目名称
        name: 实验名称
        **kwargs: 其他训练参数

Returns:
    训练结果字典
"""
# 加载预训练模型
model = YOLO(self.base_model_path)

# 设置项目名称
if project is None:
    project = self.output_dir
if name is None:
    name = f"train_{datetime.now().strftime('%Y%m%d_%H%M%S')}"

# 开始训练
results = model.train(
    data=data_yaml,
    epochs=epochs,
    imgsz=img_size,
    batch=batch_size,
    device=device,
    project=project,
    name=name,
    # 数据增强参数
    hsv_h=0.015,      # 色调增强
    hsv_s=0.7,        # 饱和度增强
    hsv_v=0.4,        # 亮度增强
    degrees=0.0,      # 旋转角度
    translate=0.1,    # 平移
    scale=0.5,        # 缩放
   fliplr=0.5,        # 水平翻转
    mosaic=1.0,       # Mosaic增强
    mixup=0.1,        # MixUp增强
    # 保存参数
    save=True,
    save_period=10,   # 每10轮保存一次
    # 其他参数
    patience=50,      # 早停耐心值
    plots=True,       # 生成可视化图表
    **kwargs
)

# 获取最佳模型路径
best_model_path = Path(project) / name / "weights" / "best.pt"
last_model_path = Path(project) / name / "weights" / "last.pt"

# 读取训练指标
results_csv = Path(project) / name / "results.csv"

```

```

metrics = self._parse_results_csv(results_csv)

return {
    "best_model_path": str(best_model_path),
    "last_model_path": str(last_model_path),
    "metrics": metrics,
    "training_dir": str(Path(project) / name),
}

def _parse_results_csv(self, csv_path: Path) -> dict:
    """解析训练结果csv文件"""
    import pandas as pd

    if not csv_path.exists():
        return {}

    df = pd.read_csv(csv_path)

    # 获取最终指标
    metrics = {
        "epochs_completed": len(df),
        "final_map50": float(df["metrics/mAP50(B)"].iloc[-1]) if "metrics/mAP50(B)" in
df.columns else 0,
        "final_map50_95": float(df["metrics/mAP50-95(B)"].iloc[-1]) if "metrics/mAP50-
95(B)" in df.columns else 0,
        "final_precision": float(df["metrics/precision(B)"].iloc[-1]) if
"metrics/precision(B)" in df.columns else 0,
        "final_recall": float(df["metrics/recall(B)"].iloc[-1]) if "metrics/recall(B)"
in df.columns else 0,
        "best_map50": float(df["metrics/mAP50(B)"].max()) if "metrics/mAP50(B)" in
df.columns else 0,
    }

    return metrics

def validate(self, model_path: str, data_yaml: str, device: str = "0") -> dict:
    """
    验证模型

    Args:
        model_path: 模型文件路径
        data_yaml: 数据集配置文件路径
        device: 验证设备

    Returns:
        验证结果字典
    """
    model = YOLO(model_path)
    results = model.val(data=data_yaml, device=device)

    return {
        "map50": float(results.box.map50),
    }

```

```

        "map50_95": float(results.box.map),
        "precision": float(results.box.mp),
        "recall": float(results.box.mr),
        "per_class_ap": {
            model.names[i]: float(ap)
            for i, ap in enumerate(results.box.ap50)
        }
    }

def export_model(self, model_path: str, format: str = "onnx", device: str = "0") ->
str:
    """
    导出模型为其他格式

    Args:
        model_path: 模型文件路径
        format: 导出格式 (onnx, torchscript, tensorflow, etc.)
        device: 设备

    Returns:
        导出模型路径
    """
    model = YOLO(model_path)
    exported_path = model.export(format=format)
    return exported_path

```

## 5.8 训练配置管理

```

# backend/app/config/training.py
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class TrainingConfig:
    """训练配置"""
    # 基础配置
    base_model: str = "models/yolov11n.pt" # 预训练模型
    output_dir: str = "runs/train"

    # 训练参数
    epochs: int = 100
    img_size: int = 640
    batch_size: int = 16
    device: str = "0"

    # 学习率配置
    lr0: float = 0.01 # 初始学习率
    lrf: float = 0.01 # 最终学习率 (相对于初始学习率)

    # 优化器配置
    optimizer: str = "SGD" # SGD, Adam, AdamW

```

```

# 数据增强
hsv_h: float = 0.015      # 色调增强
hsv_s: float = 0.7        # 饱和度增强
hsv_v: float = 0.4        # 亮度增强
degrees: float = 0.0      # 旋转角度
translate: float = 0.1    # 平移
scale: float = 0.5        # 缩放
fliplr: float = 0.5       # 水平翻转概率
mosaic: float = 1.0       # Mosaic增强概率
mixup: float = 0.1        # MixUp增强概率

# 早停配置
patience: int = 50       # 早停耐心值

# 保存配置
save_period: int = 10     # 每N轮保存一次

@classmethod
def from_dict(cls, config_dict: dict) -> "TrainingConfig":
    """从字典创建配置"""
    return cls(**{k: v for k, v in config_dict.items() if hasattr(cls, k)})

```

## 5.9 训练API接口

```

# backend/app/api/training.py
from fastapi import APIRouter, UploadFile, File, Form, BackgroundTasks
from fastapi.responses import FileResponse
from pydantic import BaseModel
from typing import Optional
import shutil
import os
from pathlib import Path

from app.training.training_service import TrainingService
from app.training.data_converter import DataConverter

router = APIRouter(prefix="/api/training", tags=["模型训练"])

training_service = TrainingService()

class TrainRequest(BaseModel):
    """训练请求"""
    scene_name: str          # 场景名称
    data_yaml: str           # 数据集配置文件路径
    model_name: str = "yolov11n" # 模型名称
    epochs: int = 100        # 训练轮数
    img_size: int = 640      # 图像尺寸
    batch_size: int = 16     # 批次大小
    device: str = "0"        # 训练设备

```

```

class TrainResponse(BaseModel):
    """训练响应"""
    task_id: str
    status: str
    message: str

# 存储训练任务状态
training_tasks = {}

@router.post("/start")
async def start_training(
    request: TrainRequest,
    background_tasks: BackgroundTasks
):
    """启动训练任务"""
    import uuid

    task_id = str(uuid.uuid4())
    training_tasks[task_id] = {
        "status": "pending",
        "progress": 0,
        "scene_name": request.scene_name,
    }

# 在后台执行训练
def run_training():
    try:
        training_tasks[task_id]["status"] = "running"

        result = training_service.train(
            data_yaml=request.data_yaml,
            model_name=request.model_name,
            epochs=request.epochs,
            img_size=request.img_size,
            batch_size=request.batch_size,
            device=request.device,
            project=f"runs/train/{request.scene_name}",
        )

        training_tasks[task_id].update({
            "status": "completed",
            "progress": 100,
            "result": result,
        })
    except Exception as e:
        training_tasks[task_id].update({
            "status": "failed",
            "error": str(e),
        })

background_tasks.add_task(run_training)

```

```

    return TrainResponse(
        task_id=task_id,
        status="pending",
        message="训练任务已启动"
    )

@router.get("/status/{task_id}")
async def get_training_status(task_id: str):
    """获取训练任务状态"""
    if task_id not in training_tasks:
        return {"error": "Task not found"}
    return training_tasks[task_id]

@router.post("/upload_dataset")
async def upload_dataset(
    scene_name: str = Form(...),
    file: UploadFile = File(...),
):
    """上传数据集 (ZIP格式) """
    upload_dir = Path("datasets") / scene_name
    upload_dir.mkdir(parents=True, exist_ok=True)

    # 保存ZIP文件
    zip_path = upload_dir / "dataset.zip"
    with open(zip_path, "wb") as f:
        shutil.copyfileobj(file.file, f)

    # 解压ZIP文件
    import zipfile
    with zipfile.ZipFile(zip_path, "r") as zip_ref:
        zip_ref.extractall(upload_dir)

    # 删除ZIP文件
    zip_path.unlink()

    return {
        "message": "数据集上传成功",
        "scene_name": scene_name,
        "path": str(upload_dir),
    }

@router.post("/convert_format")
async def convert_dataset_format(
    source_format: str = Form(...), # voc, coco, labelme
    source_dir: str = Form(...),
    output_dir: str = Form(...),
    class_mapping: str = Form(...), # JSON格式的类别映射
):
    """转换数据集格式"""
    import json

    mapping = json.loads(class_mapping)

```

```

if source_format == "voc":
    DataConverter.voc_to_yolo(source_dir, output_dir, mapping)
elif source_format == "coco":
    # COCO需要JSON文件路径
    json_files = list(Path(source_dir).glob("*.json"))
    if json_files:
        DataConverter.coco_to_yolo(str(json_files[0]), source_dir, output_dir)
elif source_format == "labelme":
    DataConverter.labelme_to_yolo(source_dir, output_dir, mapping)
else:
    return {"error": f"Unsupported format: {source_format}"}

return {
    "message": "格式转换成功",
    "output_dir": output_dir,
}

@router.get("/download_model/{scene_name}")
async def download_model(scene_name: str):
    """下载训练好的模型"""
    model_path = Path(f"runs/train/{scene_name}") / "weights" / "best.pt"

    if not model_path.exists():
        return {"error": "Model not found"}

    return FileResponse(
        model_path,
        media_type="application/octet-stream",
        filename=f"{scene_name}_best.pt"
    )

@router.post("/validate")
async def validate_model(
    model_path: str = Form(...),
    data_yaml: str = Form(...),
    device: str = Form("0"),
):
    """验证模型"""
    results = training_service.validate(model_path, data_yaml, device)
    return results

```

## 5.10 前端训练界面

```

<!-- frontend/src/views/TrainingPage.vue -->
<template>
  <div class="training-page">
    <el-card class="training-card">
      <template #header>
        <div class="card-header">
          <span>模型训练</span>

```

```

    </div>
</template>

<!-- 训练配置表单 -->
<el-form :model="trainForm" label-width="120px">
  <el-form-item label="场景名称">
    <el-input v-model="trainForm.scene_name" placeholder="如: remote_sensing" />
  </el-form-item>

  <el-form-item label="数据集配置">
    <el-input v-model="trainForm.data_yaml" placeholder="datasets/xxx/data.yaml" />
  </el-form-item>

  <el-form-item label="模型选择">
    <el-select v-model="trainForm.model_name">
      <el-option label="YOLOv11n (轻量)" value="yolov11n" />
      <el-option label="YOLOv11s (小型)" value="yolov11s" />
      <el-form-item label="YOLOv11m (中型)" value="yolov11m" />
      <el-option label="YOLOv11l (大型)" value="yolov11l" />
      <el-option label="YOLOv11x (超大)" value="yolov11x" />
    </el-select>
  </el-form-item>

  <el-form-item label="训练轮数">
    <el-slider v-model="trainForm.epochs" :min="10" :max="500" :step="10" show-input
/>
  </el-form-item>

  <el-form-item label="图像尺寸">
    <el-select v-model="trainForm.img_size">
      <el-option :value="320" label="320" />
      <el-option :value="416" label="416" />
      <el-option :value="640" label="640 (推荐)" />
      <el-option :value="800" label="800" />
      <el-option :value="1024" label="1024" />
    </el-select>
  </el-form-item>

  <el-form-item label="批次大小">
    <el-slider v-model="trainForm.batch_size" :min="1" :max="64" :step="1" show-
input />
  </el-form-item>

  <el-form-item label="训练设备">
    <el-radio-group v-model="trainForm.device">
      <el-radio label="0">GPU 0</el-radio>
      <el-radio label="1">GPU 1</el-radio>
      <el-radio label="cpu">CPU</el-radio>
    </el-radio-group>
  </el-form-item>

  <el-form-item>

```



```

        <el-button type="primary" @click="startTraining" :loading="training">
            开始训练
        </el-button>
    </el-form-item>
</el-form>
</el-card>

<!-- 训练状态 -->
<el-card v-if="taskId" class="status-card">
    <template #header>
        <div class="card-header">
            <span>训练状态</span>
            <el-tag :type="statusType">{{ taskStatus.status }}</el-tag>
        </div>
    </template>

    <el-progress :percentage="taskStatus.progress || 0" :status="progressStatus" />

    <div v-if="taskStatus.result" class="training-result">
        <h4>训练结果</h4>
        <p>最佳mAP50: {{ (taskStatus.result.metrics.best_map50 * 100).toFixed(2) }}%</p>
        <p>最终mAP50: {{ (taskStatus.result.metrics.final_map50 * 100).toFixed(2) }}%</p>
        <p>精确率: {{ (taskStatus.result.metrics.final_precision * 100).toFixed(2) }}%</p>
        <p>召回率: {{ (taskStatus.result.metrics.final_recall * 100).toFixed(2) }}%</p>

        <el-button type="success" @click="downloadModel">下载模型</el-button>
    </div>
</el-card>
</div>
</template>

<script setup>
import { ref, computed, onMounted, onUnmounted } from 'vue'
import { ElMessage } from 'element-plus'
import axios from 'axios'

const trainForm = ref({
    scene_name: '',
    data_yaml: '',
    model_name: 'yolov11n',
    epochs: 100,
    img_size: 640,
    batch_size: 16,
    device: '0',
})

const training = ref(false)
const taskId = ref(null)
const taskStatus = ref({})
let statusTimer = null

const statusType = computed(() => {

```

```

switch (taskStatus.value.status) {
  case 'completed': return 'success'
  case 'failed': return 'danger'
  case 'running': return 'warning'
  default: return 'info'
}
})

const progressStatus = computed(() => {
  if (taskStatus.value.status === 'completed') return 'success'
  if (taskStatus.value.status === 'failed') return 'exception'
  return ''
})

const startTraining = async () => {
  try {
    training.value = true
    const response = await axios.post('/api/training/start', trainForm.value)
    taskId.value = response.data.task_id

    // 开始轮询状态
    statusTimer = setInterval(fetchStatus, 5000)

    ElMessage.success('训练任务已启动')
  } catch (error) {
    ElMessage.error('启动训练失败: ' + error.message)
  } finally {
    training.value = false
  }
}

const fetchStatus = async () => {
  if (!taskId.value) return

  try {
    const response = await axios.get(`/api/training/status/${taskId.value}`)
    taskStatus.value = response.data

    // 训练完成或失败时停止轮询
    if (['completed', 'failed'].includes(response.data.status)) {
      clearInterval(statusTimer)
    }
  } catch (error) {
    console.error('获取状态失败:', error)
  }
}

const downloadModel = () => {
  window.open(`/api/training/download_model/${trainForm.value.scene_name}`)
}

onUnmounted(() => {

```

```
if (statusTimer) {
    clearInterval(statusTimer)
}
})
</script>
```

## 5.11 支持全部检测模式

训练完成后，模型需支持以下全部检测模式：

| 检测模式  | 说明          | 实现方式                              |
|-------|-------------|-----------------------------------|
| 单图检测  | 上传单张图像进行检测  | /api/detection/single             |
| 批量检测  | 上传多张图像批量检测  | /api/detection/batch              |
| 文件夹检测 | 上传整个文件夹进行检测 | /api/detection/folder             |
| 视频检测  | 上传视频逐帧检测    | /api/detection/video              |
| 摄像头检测 | 接入摄像头实时检测   | /api/detection/camera (WebSocket) |

```
# backend/app/api/detection.py
from fastapi import APIRouter, UploadFile, File, WebSocket, WebSocketDisconnect
from fastapi.responses import StreamingResponse
import cv2
import numpy as np
from pathlib import Path

from app.services.detection_service import DetectionService

router = APIRouter(prefix="/api/detection", tags=["目标检测"])
detection_service = DetectionService()

@router.post("/single")
async def detect_single(file: UploadFile = File(...)):
    """单图检测"""
    # 保存上传文件
    image_path = save_upload_file(file)

    # 执行检测
    result = detection_service.detect_single_image(image_path)

    return {
        "code": 200,
        "data": result
    }

@router.post("/batch")
async def detect_batch(files: list[UploadFile] = File(...)):
    """批量检测"""
```

```

image_paths = [save_upload_file(f) for f in files]
results = detection_service.detect_batch_images(image_paths)

return {
    "code": 200,
    "data": results
}

@router.post("/folder")
async def detect_folder(files: list[UploadFile] = File(...)):
    """文件夹检测（前端打包上传）"""
    # 与批量检测类似，但保持文件夹结构
    results = []
    for file in files:
        image_path = save_upload_file(file)
        result = detection_service.detect_single_image(image_path)
        result["filename"] = file.filename
        results.append(result)

    return {
        "code": 200,
        "data": results
    }

@router.post("/video")
async def detect_video(file: UploadFile = File(...)):
    """视频检测"""
    # 保存视频文件
    video_path = save_upload_file(file)

    # 执行视频检测
    result = detection_service.detect_video(video_path)

    return {
        "code": 200,
        "data": result
    }

@router.websocket("/camera")
async def camera_detection(websocket: WebSocket):
    """摄像头实时检测"""
    await websocket.accept()

    # 打开摄像头
    cap = cv2.VideoCapture(0)

    try:
        while True:
            # 读取帧
            ret, frame = cap.read()
            if not ret:
                break

```

```
# 执行检测
result = detection_service.detect_frame(frame)

# 发送结果
await websocket.send_json(result)

# 控制帧率
await asyncio.sleep(0.03) # ~30fps

except WebSocketDisconnect:
    pass
finally:
    cap.release()
```

## 六、YOLOv11检测模型集成

### 6.1 检测流程

用户输入图像



图像预处理

- 读取图像 (OpenCV)
- 图像尺寸调整 (640×640)
- 归一化 (0-1)
- 通道转换 (BGR→RGB)



YOLOv11 模型推理

- 模型文件: yolov11n.pt
- 推理设备: CPU / CUDA
- 置信度阈值: 0.25
- NMS阈值: 0.45



后处理

- NMS非极大值抑制
- 置信度过滤
- 边界框坐标提取
- 类别映射 (class\_id → 类别名称)



结果封装与存储

- └─ 封装 DetectionResult
- └─ 保存检测记录到 PostgreSQL
- └─ 上传标注图像到 MinIO

## 6.2 检测服务

```
# backend/app/services/detection_service.py
from ultralytics import YOLO
from app.config.detection import DetectionConfig
from app.storage.minio_client import MinIOClient
from app.database.session import get_db
from app.entity.db_models import DetectionRecord, DetectionResultRecord
from datetime import datetime
from uuid import uuid4

class DetectionService:
    """YOLOv11 检测服务"""

    def __init__(self):
        self.model = YOLO(DetectionConfig.model_path)
        self.conf_threshold = DetectionConfig.conf_threshold
        self.iou_threshold = DetectionConfig.iou_threshold
        self.device = DetectionConfig.device
        self.minio = MinIOClient()

    def detect_single_image(self, image_path: str, user_id: int) -> dict:
        """单图检测"""
        # 1. 执行推理
        results = self.model(
            image_path,
            conf=self.conf_threshold,
            iou=self.iou_threshold,
            device=self.device
        )

        # 2. 解析结果
        result = results[0]
        detected_objects = []

        for box in result.bboxes:
            obj = {
                "class_id": int(box.cls[0]),
                "class_name": self.model.names[int(box.cls[0])],
                "confidence": round(float(box.conf[0]), 4),
                "bbox": [round(v, 2) for v in box.xyxy[0].tolist()]
            }
            detected_objects.append(obj)

        # 3. 生成标注图像
```

```

annotated_image = result.plot()

# 4. 封装结果
detection_result = {
    "total_objects": len(detected_objects),
    "objects": detected_objects,
    "inference_time": round(result.speed['inference'], 2),
    "image_size": {"width": result.orig_shape[1], "height": result.orig_shape[0]}
}

# 5. 保存到数据库
self._save_to_db(user_id, image_path, detection_result)

# 6. 上传标注图像到MinIO
annotated_url = self._upload_annotated_image(user_id, annotated_image)
detection_result["annotated_image_url"] = annotated_url

return detection_result

def detect_batch_images(self, image_paths: list, user_id: int) -> list:
    """批量检测"""
    results = []
    for path in image_paths:
        result = self.detect_single_image(path, user_id)
        results.append(result)
    return results

def _save_to_db(self, user_id: int, image_path: str, result: dict):
    """保存检测结果到数据库"""
    db = next(get_db())
    record = DetectionRecord(
        user_id=user_id,
        image_path=image_path,
        total_objects=result["total_objects"],
        inference_time=result["inference_time"],
        created_at=datetime.now()
    )
    db.add(record)
    db.commit()

def _upload_annotated_image(self, user_id: int, image_bytes: bytes) -> str:
    """上传标注图像到MinIO"""
    object_name = f"detection/{user_id}/{uuid4()}.jpg"
    self.minio.upload_bytes(object_name, image_bytes)
    return self.minio.get_presigned_url(object_name)

```

## 6.3 检测配置

```
# backend/app/config/detection.py
class DetectionConfig:
    """检测配置"""
    model_path = "models/yolov11n.pt"      # 模型文件路径
    conf_threshold = 0.25                   # 置信度阈值
    iou_threshold = 0.45                   # NMS的IoU阈值
    device = "cuda"                        # 推理设备: cuda / cpu
    image_size = 640                       # 输入图像尺寸
    max_batch_size = 50                    # 批量检测最大数量
```

## 5.4 检测工具封装（Agent调用）

```
# backend/app/agent/tools/detection_tool.py
from langchain_core.tools import tool
from app.services.detection_service import DetectionService

detection_service = DetectionService()

@tool
def detect_objects(image_path: str) -> str:
    """检测遥感图像中的目标（飞机、油罐、立交桥、操场）
    输入图像路径，返回检测到的目标列表及置信度

    Args:
        image_path: 图像文件路径

    Returns:
        检测结果的自然语言描述
    """
    try:
        result = detection_service.detect_single_image(image_path, user_id=1)

        description = f"检测完成，共发现 {result['total_objects']} 个目标: \n"
        for obj in result["objects"]:
            description += f"- {obj['class_name']}, 置信度 {obj['confidence']:.2f}\n"
        description += f"推理耗时: {result['inference_time']:.2f}ms"

        return description
    except Exception as e:
        return f"检测失败: {str(e)}, 请检查图像格式是否正确"
```

# 七、各Agent实现

## 7.1 DetectionAgent

```
# backend/app/agent/agents/detection_agent.py
from langchain_core.messages import SystemMessage
from app.agent.tools.detection_tool import detect_objects
```



```

class DetectionAgent:
    """检测Agent - 专注遥感图像目标检测"""

    SYSTEM_PROMPT = """你是遥感图像目标检测专家。
    你的职责：
    - 调用检测工具识别图像中的目标（飞机、油罐、立交桥、操场）
    - 返回检测结果，包含目标类别、数量、置信度
    - 用简洁专业的中文描述检测结果
    - 如果用户未提供图像，引导用户上传
    """

    def __init__(self, llm):
        self.llm = llm
        self.tools = [detect_objects]
        self.llm_with_tools = llm.bind_tools(self.tools)

    def invoke(self, state: dict) -> dict:
        messages = [SystemMessage(content=self.SYSTEM_PROMPT)] + state["messages"]
        response = self.llm_with_tools.invoke(messages)
        # 处理工具调用...
        return {"detection_result": result, "messages": state["messages"] + [response]}

```

## 6.2 AnalysisAgent

```

# backend/app/agent/agents/analysis_agent.py
from langchain_core.messages import SystemMessage
from app.agent.tools.analysis_tool import query_history, get_statistics

class AnalysisAgent:
    """分析Agent - 专注检测数据统计与查询"""

    SYSTEM_PROMPT = """你是检测数据分析专家。
    你的职责：
    - 查询用户检测历史记录
    - 统计分析检测数据（目标数量趋势、类别分布等）
    - 用数据驱动的方式回答用户问题
    """

    def __init__(self, llm):
        self.llm = llm
        self.tools = [query_history, get_statistics]
        self.llm_with_tools = llm.bind_tools(self.tools)

    def invoke(self, state: dict) -> dict:
        messages = [SystemMessage(content=self.SYSTEM_PROMPT)] + state["messages"]
        response = self.llm_with_tools.invoke(messages)
        return {"analysis_result": result, "messages": state["messages"] + [response]}

```

## 6.3 QAAgent

```
# backend/app/agent/agents/qa_agent.py
from langchain_core.messages import SystemMessage
from app.agent.tools.knowledge_tool import search_knowledge

class QAAgent:
    """问答Agent - 专注遥感领域知识问答"""

    SYSTEM_PROMPT = """你是遥感目标检测领域知识专家。
    你的职责：
    - 回答遥感、目标检测相关的专业知识问题
    - 检索知识库中的相关内容
    - 用通俗易懂的语言解释专业概念
    """

    def __init__(self, llm):
        self.llm = llm
        self.tools = [search_knowledge]
        self.llm_with_tools = llm.bind_tools(self.tools)

    def invoke(self, state: dict) -> dict:
        messages = [SystemMessage(content=self.SYSTEM_PROMPT)] + state["messages"]
        response = self.llm_with_tools.invoke(messages)
        return {"qa_result": result, "messages": state["messages"] + [response]}
```

## 八、工具定义

```
# === DetectionAgent 专属工具 ===
# backend/app/agent/tools/detection_tool.py
@tool
def detect_objects(image_path: str) -> str:
    """检测遥感图像中的目标（飞机、油罐、立交桥、操场）
    输入图像路径，返回检测到的目标列表及置信度"""
    result = detection_service.detect_single_image(image_path, user_id)
    return format_result(result)

# === AnalysisAgent 专属工具 ===
# backend/app/agent/tools/analysis_tool.py
@tool
def query_detection_history(user_id: str, target_type: str = None) -> str:
    """查询用户的检测历史记录
    可按目标类型筛选，返回检测时间、结果摘要"""
    records = history_service.query(user_id, target_type)
    return format_history(records)

@tool
def get_detection_statistics(user_id: str) -> str:
    """获取用户的检测统计数据
    返回检测总次数、各类目标数量、最近检测时间等"""
    stats = history_service.get_statistics(user_id)
```

```

        return format_statistics(stats)

# === QAAgent 专属工具 ===
# backend/app/agent/tools/knowledge_tool.py
@tool
def search_knowledge(query: str) -> str:
    """搜索遥感目标检测知识库
    返回与查询相关的知识内容"""
    results = retriever.search(query, top_k=5)
    return format_knowledge(results)

```

## 九、数据库设计

### 9.1 数据模型

```

# backend/app/entity/db_models.py
from sqlalchemy import Column, Integer, String, Float, DateTime, ForeignKey, JSON, Text
from sqlalchemy.orm import relationship
from app.database.session import Base

class User(Base):
    """用户表"""
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, autoincrement=True)
    username = Column(String(50), unique=True, nullable=False)
    email = Column(String(100), unique=True, nullable=False)
    hashed_password = Column(String(255), nullable=False)
    created_at = Column(DateTime, default=datetime.now)

    detection_records = relationship("DetectionRecord", back_populates="user")

class DetectionRecord(Base):
    """检测记录表"""
    __tablename__ = "detection_records"

    id = Column(Integer, primary_key=True, autoincrement=True)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    image_path = Column(String(500), nullable=False)
    annotated_image_path = Column(String(500))
    total_objects = Column(Integer, default=0)
    inference_time = Column(Float)
    created_at = Column(DateTime, default=datetime.now)

    user = relationship("User", back_populates="detection_records")
    results = relationship("DetectionResultDetail", back_populates="record")

class DetectionResultDetail(Base):

```

```

"""检测结果详情表"""
__tablename__ = "detection_result_details"

id = Column(Integer, primary_key=True, autoincrement=True)
record_id = Column(Integer, ForeignKey("detection_records.id"), nullable=False)
class_name = Column(String(50), nullable=False)
confidence = Column(Float, nullable=False)
bbox = Column(JSON) # [x1, y1, x2, y2]

record = relationship("DetectionRecord", back_populates="results")

class ChatHistory(Base):
    """对话历史表"""
    __tablename__ = "chat_history"

    id = Column(Integer, primary_key=True, autoincrement=True)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    session_id = Column(String(100), nullable=False)
    role = Column(String(20), nullable=False) # user / assistant
    content = Column(Text, nullable=False)
    agent_used = Column(String(50)) # detection / analysis / qa
    created_at = Column(DateTime, default=datetime.now)

```

## 9.2 数据库连接

```

# backend/app/database/session.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
from app.config.settings import Settings

settings = Settings()

engine = create_engine(
    f"postgresql://{settings.DB_USER}:{settings.DB_PASSWORD}@{settings.DB_HOST}:
{settings.DB_PORT}/{settings.DB_NAME}",
    pool_size=10,
    max_overflow=20
)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

# 十、API接口设计

## 10.1 智能体API

```
# backend/app/api/agent.py
from fastapi import APIRouter, UploadFile, File, Depends, HTTPException
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
from jose import JWTError, jwt
from app.config.settings import settings
from app.database.session import get_db
from app.entity.db_models import User
from sqlalchemy.orm import Session
import os
import uuid

router = APIRouter(prefix="/api/agent", tags=["智能体"])

class ChatRequest(BaseModel):
    message: str
    session_id: str = None

# 依赖：获取当前用户
async def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db)
) -> User:
    """从JWT token中解析用户信息"""
    credentials_exception = HTTPException(
        status_code=401,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, settings.JWT_SECRET, algorithms=
[settings.JWT_ALGORITHM])
        user_id: int = payload.get("sub")
        if user_id is None:
            raise credentials_exception
    except JWTError:
        raise credentials_exception

    user = db.query(User).filter(User.id == user_id).first()
    if user is None:
        raise credentials_exception
    return user

# 工具函数：保存上传文件
def save_upload_file(file: UploadFile) -> str:
    """保存上传文件到临时目录，返回文件路径"""
    upload_dir = "uploads"
```

```

os.makedirs(upload_dir, exist_ok=True)

file_extension = os.path.splitext(file.filename)[1]
file_name = f"{uuid.uuid4()}{file_extension}"
file_path = os.path.join(upload_dir, file_name)

with open(file_path, "wb") as f:
    content = file.file.read()
    f.write(content)

return file_path

@router.post("/chat")
async def chat(request: ChatRequest, current_user: User = Depends(get_current_user)):
    """智能对话 - SSE流式响应"""
    async def generate():
        async for chunk in agent_graph.astream_events(
            {"messages": [HumanMessage(content=request.message)], "user_id":
current_user.id},
            config={}
        ):
            yield f"data: {json.dumps(chunk)}\n\n"
            yield "data: [DONE]\n\n"

    return StreamingResponse(generate(), media_type="text/event-stream")

@router.post("/detect")
async def detect_with_agent(
    file: UploadFile = File(...),
    current_user: User = Depends(get_current_user)
):
    """智能检测 - 上传图像并用Agent分析"""
    image_path = save_upload_file(file)
    result = agent_graph.invoke({
        "messages": [HumanMessage(content=f"检测这张图像: {image_path}")],
        "user_id": current_user.id
    })
    return {
        "response": result["final_response"],
        "detection_result": result.get("detection_result")
    }

@router.get("/history")
async def get_chat_history(current_user: User = Depends(get_current_user)):
    """获取对话历史"""
    memory = ConversationMemory(redis_client)
    history = memory.get_history(str(current_user.id))
    return {"history": history}

```

## 9.2 检测API

```
# backend/app/api/detection.py
from fastapi import APIRouter, UploadFile, File

router = APIRouter(prefix="/api/detection", tags=["检测"])

@router.post("/single")
async def single_detection(file: UploadFile = File(...)):
    """单图检测"""
    image_path = save_upload_file(file)
    result = detection_service.detect_single_image(image_path, user_id=current_user.id)
    return {"code": 200, "data": result}

@router.post("/batch")
async def batch_detection(files: list[UploadFile] = File(...)):
    """批量检测"""
    image_paths = [save_upload_file(f) for f in files]
    results = detection_service.detect_batch_images(image_paths, user_id=current_user.id)
    return {"code": 200, "data": results}
```

## 9.3 认证API

```
# backend/app/api/auth.py
from fastapi import APIRouter
from pydantic import BaseModel

router = APIRouter(prefix="/api/auth", tags=["认证"])

class LoginRequest(BaseModel):
    username: str
    password: str

class RegisterRequest(BaseModel):
    username: str
    email: str
    password: str

@router.post("/login")
async def login(request: LoginRequest):
    """用户登录"""
    user = user_service.authenticate(request.username, request.password)
    token = create_jwt_token(user.id)
    return {"code": 200, "data": {"token": token, "username": user.username}}

@router.post("/register")
async def register(request: RegisterRequest):
    """用户注册"""
    user = user_service.create_user(request.username, request.email, request.password)
    return {"code": 200, "data": {"user_id": user.id}}
```

## 9.4 API汇总

| API                     | 方法   | 说明          |
|-------------------------|------|-------------|
| /api/auth/login         | POST | 用户登录        |
| /api/auth/register      | POST | 用户注册        |
| /api/agent/chat         | POST | 智能对话（SSE流式） |
| /api/agent/detect       | POST | 智能检测        |
| /api/agent/history      | GET  | 对话历史        |
| /api/detection/single   | POST | 单图检测        |
| /api/detection/batch    | POST | 批量检测        |
| /api/history/records    | GET  | 检测记录列表      |
| /api/history/statistics | GET  | 检测统计        |

## 十、关键设计

### 10.1 LLM配置管理

```
# backend/app/config/llm.py
import os

class LLMConfig:
    """LLM服务配置"""
    # 主力: OpenAI
    openai_api_key: str = os.getenv("OPENAI_API_KEY", "")
    openai_model: str = os.getenv("OPENAI_MODEL", "gpt-4o-mini")
    openai_base_url: str = os.getenv("OPENAI_BASE_URL", "https://api.openai.com/v1")

    # 备选: Ollama
    ollama_base_url: str = os.getenv("OLLAMA_BASE_URL", "http://localhost:11434")
    ollama_model: str = os.getenv("OLLAMA_MODEL", "qwen2.5:7b")

    # 切换策略
    use_local: bool = os.getenv("USE_LOCAL_LLM", "false").lower() == "true"
```

### 10.2 全局配置

```
# backend/app/config/settings.py
import os
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    # 数据库
    DB_HOST: str = os.getenv("DB_HOST", "localhost")
```



```

DB_PORT: int = int(os.getenv("DB_PORT", "5432"))
DB_NAME: str = os.getenv("DB_NAME", "rsod_agent")
DB_USER: str = os.getenv("DB_USER", "rsod_user")
DB_PASSWORD: str = os.getenv("DB_PASSWORD", "rsod_password")

# Redis
REDIS_HOST: str = os.getenv("REDIS_HOST", "localhost")
REDIS_PORT: int = int(os.getenv("REDIS_PORT", "6379"))

# MinIO
MINIO_ENDPOINT: str = os.getenv("MINIO_ENDPOINT", "localhost:9000")
MINIO_ACCESS_KEY: str = os.getenv("MINIO_ACCESS_KEY", "minioadmin")
MINIO_SECRET_KEY: str = os.getenv("MINIO_SECRET_KEY", "minioadmin")

# JWT
JWT_SECRET: str = os.getenv("JWT_SECRET", "your-secret-key")
JWT_ALGORITHM: str = "HS256"
JWT_EXPIRE_HOURS: int = 24

```

## 10.3 流式响应

```

# backend/app/api/agent.py
@router.post("/chat")
async def chat(request: ChatRequest):
    """智能对话 - SSE流式响应"""
    async def generate():
        async for chunk in agent_graph.astream_events(
            {"messages": [HumanMessage(content=request.message)]},
            config={}
        ):
            yield f"data: {json.dumps(chunk)}\n\n"
            yield "data: [DONE]\n\n"

    return StreamingResponse(generate(), media_type="text/event-stream")

```

## 10.4 错误处理

```

def safe_invoke(agent):
    """Agent执行安全包装"""
    def wrapper(state):
        try:
            return agent.invoke(state)
        except Exception as e:
            return {"error": f"Agent执行异常: {str(e)}", "messages": []}
    return wrapper

```

## 10.5 Prompt工程

```

# backend/app/agent/prompts.py

```

```
SUPERVISOR_ROUTING_PROMPT = """你是一个任务调度器。根据用户输入，选择最合适的Agent处理。
```

可选Agent：

- **detection**：用户需要检测遥感图像中的目标（飞机、油罐、立交桥、操场）
- **analysis**：用户需要查询检测历史、统计分析数据
- **qa**：用户提出遥感领域知识问题

如果是复合任务，返回多个Agent名称，用逗号分隔。

只回复Agent名称，不要回复其他内容。

```
"""
```

```
DETECTION_AGENT_PROMPT = """你是遥感图像目标检测专家。
```

你的职责：

- 调用检测工具识别图像中的目标
- 返回检测结果，包含目标类别、数量、置信度
- 用简洁专业的中文描述检测结果

```
"""
```

```
ANALYSIS_AGENT_PROMPT = """你是检测数据分析专家。
```

你的职责：

- 查询用户检测历史记录
- 统计分析检测数据（目标数量趋势、类别分布等）
- 用数据驱动的方式回答用户问题

```
"""
```

```
QA_AGENT_PROMPT = """你是遥感目标检测领域知识专家。
```

你的职责：

- 回答遥感、目标检测相关的专业知识问题
- 检索知识库中的相关内容
- 用通俗易懂的语言解释专业概念

```
"""
```

## 10.6 对话记忆

```
# backend/app/agent/memory.py
from langchain.memory import RedisChatMessageHistory

class ConversationMemory:
    """对话记忆管理"""

    def __init__(self, redis_client, session_ttl=3600):
        self.redis = redis_client
        self.ttl = session_ttl

    def get_history(self, user_id: str):
        """获取用户对话历史"""
        return RedisChatMessageHistory(
            session_id=f"agent:{user_id}",
            redis_url="redis://localhost:6379",
            ttl=self.ttl
```

```
)

def add_message(self, user_id: str, role: str, content: str):
    """添加对话消息"""
    history = self.get_history(user_id)
    if role == "user":
        history.add_user_message(content)
    else:
        history.add_ai_message(content)
```

## 十一、多Agent工作流程

用户输入: "帮我检测这张遥感图像, 然后分析一下最近的检测趋势"



1. Supervisor Agent 意图识别与任务拆解
  - 识别为复合任务: 需要检测 + 分析
  - 拆解为两个子任务, 按顺序执行



2. 路由到 DetectionAgent
  - DetectionAgent 调用 detect\_objects() 工具
  - YOLOv11检测图像, 返回检测结果
  - 结果写入 AgentState.detection\_result



3. 路由到 AnalysisAgent
  - AnalysisAgent 调用 get\_statistics() 工具
  - 查询历史数据, 分析检测趋势
  - 结果写入 AgentState.analysis\_result



4. Supervisor 结果汇总
  - 整合 DetectionAgent 和 AnalysisAgent 的结果
  - LLM生成自然语言综合报告
  - SSE流式返回给用户
  - 存储对话历史到Redis



输出综合分析报告给用户

# 前端展示多Agent协作过程



## 十三、依赖清单

### 13.1 后端依赖

```
# backend/requirements.txt

# Web框架
fastapi==0.104.0
uvicorn[standard]==0.24.0
python-multipart==0.0.6

# 智能体
langchain==0.3.0
langchain-openai==0.2.0
langchain-community==0.3.0
langgraph==0.2.0
openai==1.50.0

# 目标检测
```

```
ultralytics==8.3.0
opencv-python==4.8.0
Pillow==10.0.0

# 数据库
sqlalchemy==2.0.0
psycopg2-binary==2.9.0
pgvector==0.3.0

# 缓存
redis==5.0.0

# 对象存储
minio==7.2.0

# 认证
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4

# 数据验证
pydantic==2.5.0
pydantic-settings==2.1.0

# 本地LLM备选
ollama==0.3.0
```

## 12.2 前端依赖

```
// frontend/package.json
{
  "dependencies": {
    "vue": "^3.5.0",
    "vue-router": "^4.6.0",
    "pinia": "^3.0.0",
    "element-plus": "^2.7.0",
    "axios": "^1.7.0",
    "echarts": "^5.5.0",
    "markdown-it": "^14.0.0"
  },
  "devDependencies": {
    "vite": "^6.0.0",
    "@vitejs/plugin-vue": "^5.0.0"
  }
}
```

---

## 十三、部署架构

### 13.1 Docker Compose

---

```
# docker-compose.yml
version: '3.8'

services:
  # PostgreSQL + Pgvector
  postgres:
    image: pgvector/pgvector:pg15
    environment:
      POSTGRES_DB: rsod_agent
      POSTGRES_USER: rsod_user
      POSTGRES_PASSWORD: rsod_password
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  # Redis
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

  # MinIO
  minio:
    image: minio/minio:latest
    command: server /data --console-address ":9001"
    ports:
      - "9000:9000"
      - "9001:9001"
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin
    volumes:
      - minio_data:/data

  # Ollama 本地LLM (可选)
  ollama:
    image: ollama/ollama:latest
    ports:
      - "11434:11434"
    volumes:
      - ollama_data:/root/.ollama

  # 后端服务
  backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - DB_HOST=postgres
      - DB_PORT=5432
```

```

- DB_NAME=rsod_agent
- DB_USER=rsod_user
- DB_PASSWORD=rsod_password
- REDIS_HOST=redis
- REDIS_PORT=6379
- MINIO_ENDPOINT=minio:9000
- OLLAMA_BASE_URL=http://ollama:11434
volumes:
  - ./models:/app/models
depends_on:
  - postgres
  - redis
  - minio

# 前端服务
frontend:
  build: ./frontend
  ports:
    - "80:80"
  depends_on:
    - backend

volumes:
  postgres_data:
  minio_data:
  ollama_data:

```

## 13.2 环境变量

```

# .env
# OpenAI
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxx
OPENAI_MODEL=gpt-4o-mini
OPENAI_BASE_URL=https://api.openai.com/v1

# 数据库
DB_HOST=localhost
DB_PORT=5432
DB_NAME=rsod_agent
DB_USER=rsod_user
DB_PASSWORD=rsod_password

# Redis
REDIS_HOST=localhost
REDIS_PORT=6379

# MinIO
MINIO_ENDPOINT=localhost:9000
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin

```

```
# LLM
USE_LOCAL_LLM=false
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=qwen2.5:7b

# JWT
JWT_SECRET=your-secret-key-change-in-production
```

## 十四、多Agent教学价值

### 14.1 通过本项目掌握的企业级技能

| 技能领域      | 具体能力                     | 企业应用场景        |
|-----------|--------------------------|---------------|
| 多Agent架构  | Supervisor模式、Agent注册与发现  | 企业AI中台、智能客服系统 |
| 工作流编排     | LangGraph状态图、条件路由、状态传递   | 企业业务流程自动化     |
| 状态管理      | 全局状态设计、Agent间数据传递        | 分布式系统状态同步     |
| 工具权限隔离    | 每个Agent绑定专属工具集           | 企业权限管理、安全隔离   |
| 错误隔离      | 单Agent失败不影响整体系统          | 企业容错设计、服务降级   |
| 可扩展设计     | 新增Agent无需修改现有代码          | 企业插件化架构       |
| Prompt工程  | 多Agent Prompt设计、路由Prompt | 企业LLM应用核心能力   |
| 流式交互      | SSE流式输出、实时展示Agent执行过程    | 企业AI产品用户体验    |
| YOLOv11集成 | 模型加载、推理、后处理、部署           | CV算法工程师核心技能   |
| 全栈开发      | FastAPI + Vue3 + Docker  | 全栈工程师基础能力     |

### 14.2 面试高频问题覆盖



| 面试问题           | 本项目实践                           |
|----------------|---------------------------------|
| 你了解多Agent系统吗？  | Supervisor + 3个专业Agent          |
| Agent之间如何协作？   | LangGraph状态图编排                  |
| 如何做任务路由？       | Supervisor意图识别 + 条件路由           |
| Agent之间如何传递数据？ | AgentState全局状态管理                |
| 如何保证系统可扩展？     | 新增Agent注册到StateGraph即可          |
| 如何处理Agent执行失败？ | 错误隔离 + Supervisor兜底             |
| YOLOv11如何部署？   | Docker + MinIO + FastAPI        |
| 如何优化检测性能？      | GPU推理 + 批量检测 + NMS优化            |
| 如何设计流式响应？      | SSE + FastAPI StreamingResponse |
| 如何管理对话上下文？     | Redis + LangChain Memory        |

## 十六、总结

### 技术栈概览

| 维度       | 选型                   | 说明                 |
|----------|----------------------|--------------------|
| Agent框架  | LangChain            | Agent基础能力          |
| 多Agent编排 | LangGraph            | 企业级多Agent workflow |
| LLM服务    | OpenAI + Ollama      | 云端主力 + 本地备选        |
| 向量存储     | Pgvector             | 复用PostgreSQL       |
| 检测模型     | YOLOv11              | Ultralytics 8.3+   |
| Web框架    | FastAPI              | 异步高性能              |
| 前端       | Vue 3 + Element Plus | 现代前端               |
| 数据库      | PostgreSQL           | 关系型主库              |
| 缓存       | Redis                | 对话历史 + 缓存          |
| 对象存储     | MinIO                | 图片/模型存储            |
| 部署       | Docker Compose       | 容器化部署              |

### 核心价值

- **从零构建**：完整的项目结构，清晰的模块划分
- **多Agent架构**：Supervisor + 3个专业Agent，对标企业级开发模式
- **LangGraph编排**：状态图、条件路由、状态管理，掌握企业主流方案
- **YOLOv11检测**：单图/批量全场景检测能力
- **职责分离**：每个Agent独立Prompt、独立工具集、独立迭代
- **可扩展设计**：新增能力只需添加新Agent，无需修改现有代码
- **错误隔离**：单Agent失败不影响整体系统